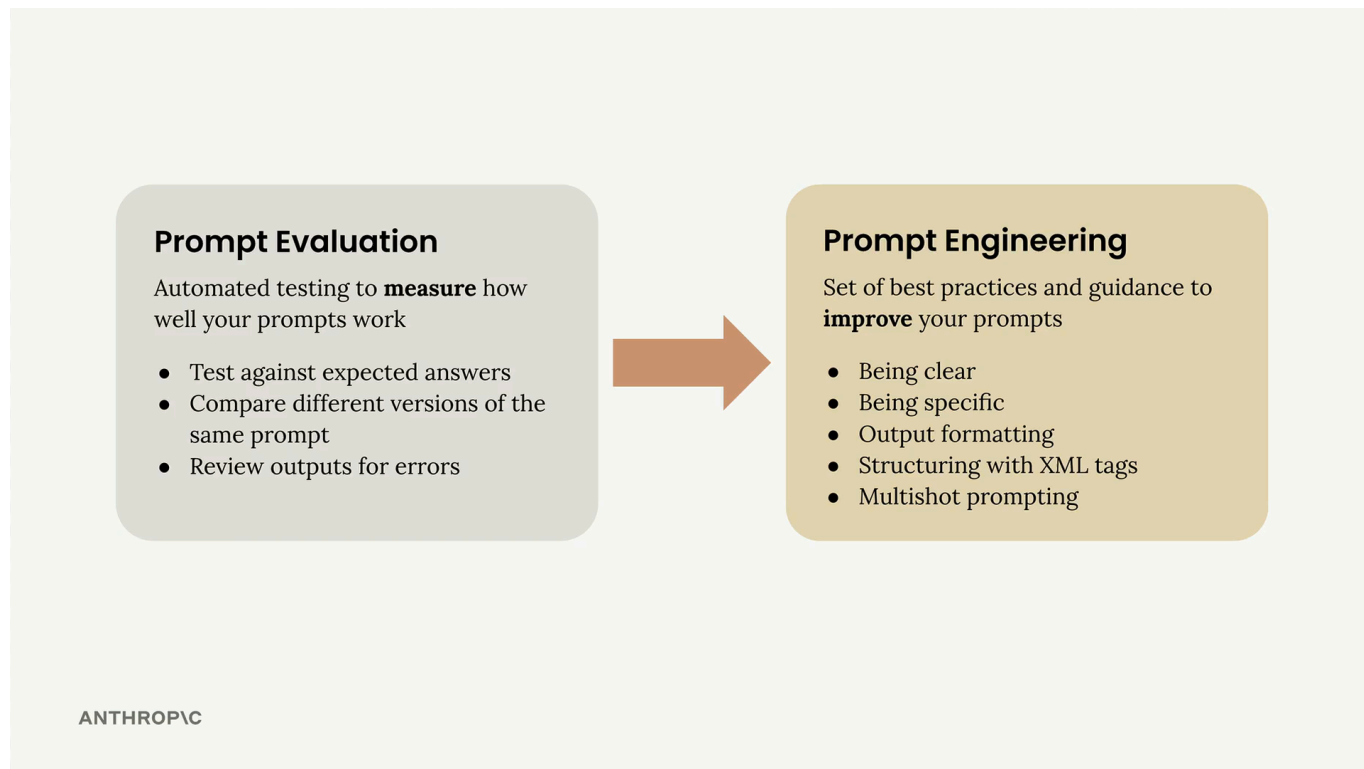


Prompt Engineering

Video Link: [Prompt Engineering](#)

Prompt engineering is about taking a prompt you've written and improving it to get more reliable, higher-quality outputs. This process involves iterative refinement - starting with a basic prompt, evaluating its performance, then systematically applying engineering techniques to improve it.



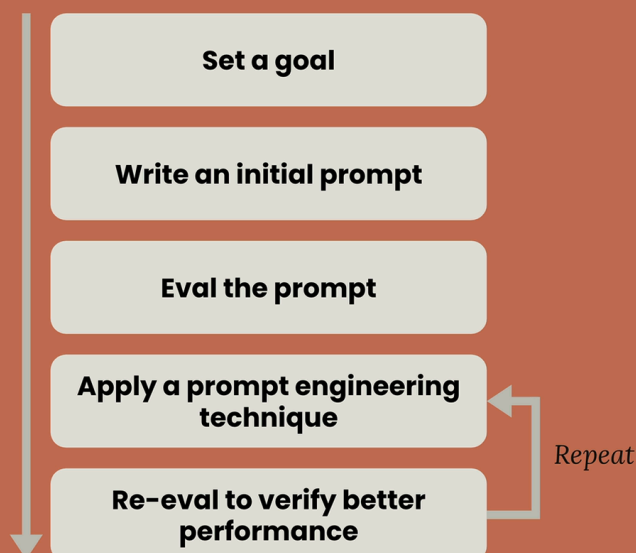
The Iterative Improvement Process

The approach follows a clear cycle that you can repeat until you achieve your desired results:

How We'll Learn Prompt Engineering

- We'll write an initial prompt, then improve it step by step while learning new techniques
- **Note:** I made an improved version of the eval pipeline, you can find it in the "001_prompting.ipynb" file. Be sure to use this file!

ANTHROPIC

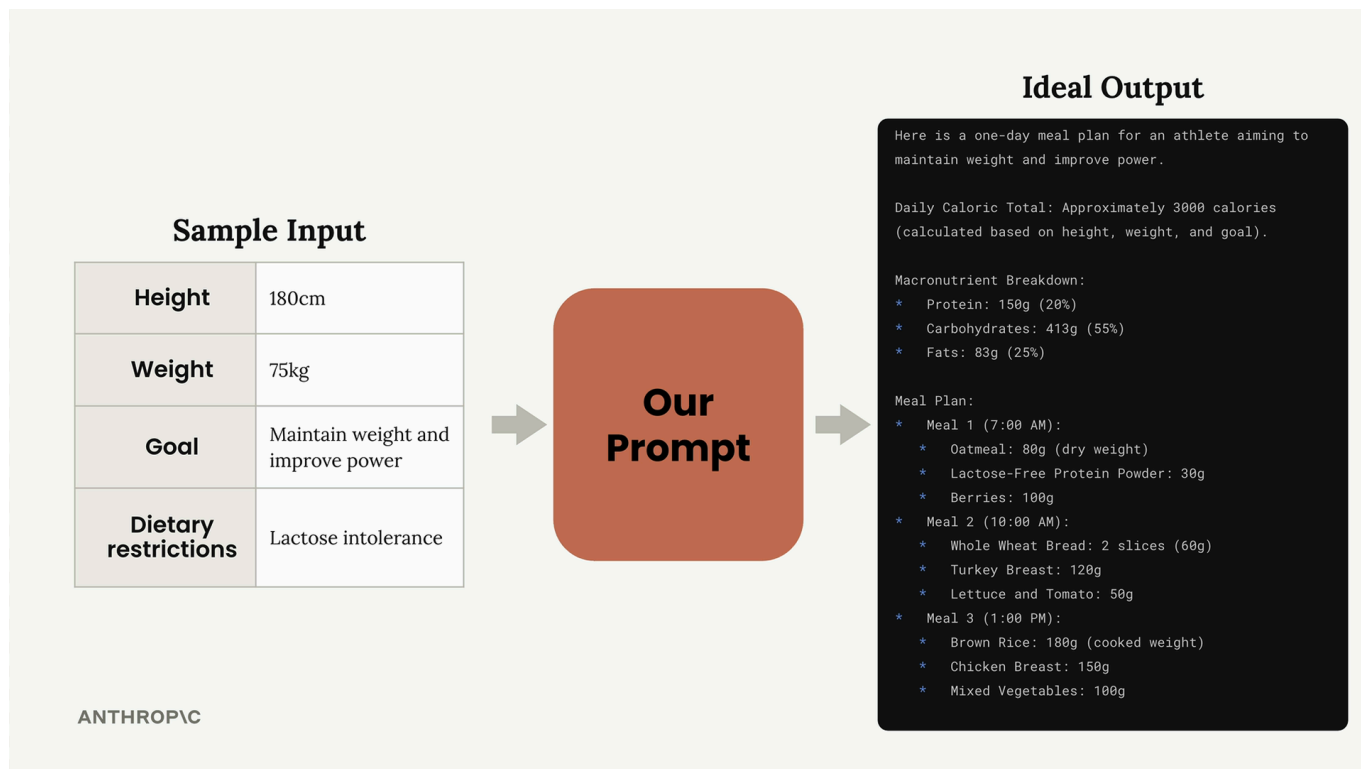


1. **Set a goal** - Define what you want your prompt to accomplish
2. **Write an initial prompt** - Create a basic first attempt
3. **Evaluate the prompt** - Test it against your criteria
4. **Apply prompt engineering techniques** - Use specific methods to improve performance
5. **Re-evaluate** - Verify that your changes actually improved the results

You repeat the last two steps until you're satisfied with the performance. Each iteration should show measurable improvement in your evaluation scores.

Setting Up Your Evaluation Pipeline

To demonstrate this process, we'll work with a practical example: creating a prompt that generates one-day meal plans for athletes. The prompt needs to take into account an athlete's height, weight, goals, and dietary restrictions, then produce a comprehensive meal plan.



The evaluation setup uses a `PromptEvaluator` class that handles dataset generation and model grading. When creating your evaluator instance, you can control concurrency with the `max_concurrent_tasks` parameter:

```
evaluator = PromptEvaluator(max_concurrent_tasks=5)
```

Start with a low concurrency value (like 3) to avoid rate limit errors. You can increase it if your API quota allows for faster processing.

Generating Test Data

The evaluation system can automatically generate test cases based on your prompt requirements. You define what inputs your prompt needs:

```
dataset = evaluator.generate_dataset(
    task_description="Write a compact, concise 1 day meal plan for a single athlete",
    prompt_inputs_spec={
        "height": "Athlete's height in cm",
        "weight": "Athlete's weight in kg",
        "goal": "Goal of the athlete",
        "restrictions": "Dietary restrictions of the athlete"
    },
    output_file="dataset.json",
    num_cases=3
)
```

Keep the number of test cases low (2-3) during development to speed up your iteration cycle. You can increase this for final validation.

Writing Your Initial Prompt

Start with a simple, naive prompt to establish a baseline. Here's an example of a deliberately basic first attempt:

```
def run_prompt(prompt_inputs):
    prompt = f"""
    What should this person eat?

    - Height: {prompt_inputs["height"]}
    - Weight: {prompt_inputs["weight"]}
    - Goal: {prompt_inputs["goal"]}
    - Dietary restrictions: {prompt_inputs["restrictions"]}
    """

    messages = []
    add_user_message(messages, prompt)
    return chat(messages)
```

This basic prompt will likely produce poor results, but it gives you a starting point to measure improvement against.

Adding Evaluation Criteria

When running your evaluation, you can specify additional criteria that the grading model should consider:

```
results = evaluator.run_evaluation(
    run_prompt_function=run_prompt,
    dataset_file="dataset.json",
    extra_criteria="""
    The output should include:
    - Daily caloric total
    - Macronutrient breakdown
    - Meals with exact foods, portions, and timing
    """
)
```

This helps ensure your prompt is evaluated against the specific requirements that matter for your use case.

Analyzing Results

After running an evaluation, you'll get both a numerical score and a detailed HTML report. The report shows you exactly how each test case performed, including the model's reasoning for each score.

Prompt Evaluation Report

Total Test Cases

50

Average Score

2.3 / 10

Pass Rate (≥7)

0.0%

Prompt Inputs	Scenario	Solution Criteria	Output	Score	Reasoning
Athlete needing increased vitamin D, male, 80kg, incorporating it	height: 180 weight: 80 goal: Increase Vitamin D intake while maintaining current fitness level restrictions: None	- Provides a 1-day meal plan with breakfast, lunch, dinner, and snacks. - Meal plan is concise and easily understandable. - Includes at least three food sources high in Vitamin D (e.g., salmon, fortified milk, eggs). - Total caloric intake is appropriate for an 80kg male athlete maintaining fitness (approximately 2500-3000 calories). - Macronutrient balance (protein, carbs, fats) is reasonable for an athlete (e.g., protein around 1.2-1.7g/kg). - Meal plan is practical and uses readily available foods. - Avoids excessive saturated fat or added sugars.	To increase Vitamin D intake while maintaining current fitness: - Fatty fish: Salmon, tuna, mackerel - Egg yolks - Fortified foods: Milk, yogurt, cereal, orange juice - Vitamin D supplement (if needed, consult a doctor) Maintain current diet with these additions.	2	The solution only lists food sources of Vitamin D without creating a comprehensive meal plan. It misses crucial details like portion sizes, timing, caloric intake, and macronutrient breakdown, failing to meet the core requirements of the task.
Athlete needing increased potassium, male, 75kg, dietary sources	height: 180 weight: 75 goal: Increase dietary potassium intake restrictions: None	- Meal plan is a 1-day plan. - Meal plan is concise and easily followed. - Meal plan is tailored for an athlete (provides sufficient calories and macronutrients). - Meal plan includes at least 3 meals (breakfast, lunch, dinner). - Meal plan includes specific food items and portion sizes. - Meal plan prioritizes potassium-rich foods (e.g., bananas, sweet potatoes, spinach, beans). - Meal plan provides a reasonable estimate of total potassium intake for the day (at least 3500mg). - Meal plan is realistic and uses readily available food items. - Instructions are clear and easy to understand.	To increase potassium intake, this person should eat more of the following: - Sweet potatoes - White potatoes (with skin) - Beans (white, kidney, lima) - Dark leafy greens (spinach, chard) - Dried apricots and prunes - Bananas - Avocados - Cantaloupe and honeydew melon - Tomatoes and tomato sauce	1	The response only provides a list of potassium-rich foods, failing to deliver a comprehensive 1-day meal plan with specific meals, portions, timing, and nutritional information as requested. It does not meet the core requirements of the task.
Older athlete, female, 60kg, maintaining bone density	height: 165 weight: 60	Provides a 1-day meal plan including breakfast, lunch, dinner, and at least two snacks	Here's a general dietary approach:	1	The response provides general dietary guidelines but fails to

Don't be discouraged by low initial scores - a score of 2.3 out of 10 is typical for a first attempt. The goal is to see consistent improvement as you apply engineering techniques.

The detailed evaluation report helps you understand exactly where your prompt is failing and what improvements are needed. Use this feedback to guide your next iteration.

Next Steps

With your baseline established, you're ready to start applying specific prompt engineering techniques. Each technique you learn should result in measurable improvement in your evaluation scores, gradually transforming your basic prompt into a reliable, high-performing tool.

Remember that prompt engineering is an iterative process. The key is to make one change at a time, evaluate the impact, and build on what works. This systematic approach ensures you understand which techniques provide the most value for your specific use case.